

taskr

Proje Şartnamesi · Teşhis + Yeniden İnşa

Sürüm 1.0 · Hazırlanma: 25 Temmuz 2026 · Uygulama: Hafta 0 - Hafta 1

Bu belge proje klasörünün kökünde saklanmalıdır: `taskr/SPEC.pdf`

1. Bu Proje Neden Var?

Bu bir öğrenme projesi değil. Bir **ölçüm aleti** ve ardından bir **yeniden inşa alıştırması**. Amaç bir görev yöneticisi yapmak değil — dünyanın buna ihtiyacı yok. Amaç, 2 yıllık aradan sonra Python bilginin **tam olarak nerede** köreldiğini ölçmek ve o boşlukları kapatmak.

1.1 Neden bir görev yöneticisi?

Bu proje bilinçli olarak seçildi. Hiçbir harici kütüphane, hiçbir alan bilgisi, hiçbir API anahtarı gerektirmiyor — ama **körelen alanların tam olarak hepsine** dokunuyor:

Alan	Projede nerede karşına çıkıyor
Veri modelleme	Görev nesnesinin tasarımı — dataclass mı, dict mi, hangi alanlar
Dosya G/Ç ve kalıcılık	JSON okuma/yazma, klasör oluşturma, bozuk dosya yönetimi
Hata yönetimi	Kendi exception hiyerarşin, kullanıcıya traceback göstermemek
CLI ayrıştırma	argparse subcommand yapısı — en çok unutulmuş konu
Tarih/saat	Parse etme, karşılaştırma, serileştirme
Paket yapısı	Çok modüllü paket, <code>__main__.py</code> , import düzeni
Tip sistemi	Tüm imzalarda type hint, <code>mypy --strict</code> uyumu
Test	pytest, fixture, geçici dosya sistemi, kenar durumlar

Bir web uygulaması bu alanların yarısını gizler (framework halleder). Bir algoritma sorusu hiçbirine dokunmaz. Bu proje hepsini açığa çıkarır.

1.2 İki fazlı yapı — en kritik tasarım kararı

	FAZ 0 — <code>taskr-v0</code>	FAZ 1 — <code>taskr</code>
Ne zaman	Hafta 0, Cumartesi	Hafta 1, Cumartesi + Pazar
Süre	3 saat (kesin sınır)	10 saat
Amaç	Ölçmek	İnşa etmek
Kalite beklentisi	Yok. Berbat olabilir.	Yüksek. Testli, tipli, dokümante.
Yayınlanır mı	Hayır. Yerelde kalır, commit edilmez.	Evet. GitHub'da public.
Çıktısı	<code>hatalar.md</code> = kişisel müfredatın	Portföyündeki ilk bitmiş proje

İki kez yazmak neden şart?

Birinci yazım *ne kadar köreldiğini* gösterir. İkinci yazım *geri getirir*. Ve aradaki farkı bizzat görmek — aynı problemi 3 saatte zar zor çözerken, bir hafta sonra temiz biçimde çözmek — planın devamı için sahip olacağın en güçlü motivasyon kaynağıdır. Bu farkı hissetmeden devam etme.

2. FAZ 0 — Teşhis Oturumu

Hafta 0 · Cumartesi · 3 saat · Tek oturum

2.1 Kurallar — istisnasız

Kural	Detay	Neden
Süre	3 saat. Zamanlayıcı kur. Bitmese de dur.	Ölçüm sabit koşulda yapılır
Yapay zekâ	Kapalı. ChatGPT, Claude, Copilot, Gemini — hiçbiri.	AI kullanırsan neyi bilmediğini asla öğrenemezsin
İnternet	Sadece docs.python.org . Stack Overflow yok, blog yok, YouTube yok.	Resmî doküman okumak zaten kazanılması gereken bir beceri
Kütüphane	Sadece standart kütüphane. pip install yok.	Bağımlılık, eksiği gizler
Eski kod	Kendi eski projelerine bakma.	Kopyalamak hatırlamak değildir
Not defteri	Yanında açık. Takıldığın her anı anında yaz.	Seans bitince hatırlamazsın

Takıldığında ne yapacaksın? Bu en önemli kural.

1. Takıldığın anı not defterine yaz (kod harfi + tek cümle)
2. **2 dakika** kendi başına düşün
3. Çözemezsen docs.python.org 'a bak
4. Yine çözemezsen **o kısmı atla**, # TODO: bilmiyorum yaz, devam et

Takılmak **başarısızlık değil, veridir**. Projenin asıl ürünü çalışan kod değil, takılma listesidir.

2.2 Faz 0 zaman planı

09:00 – 09:10 Şartnameyi oku (Bölüm 3)
09:10 – 12:10 Kod yaz. Zamanlayıcı çalışıyor.
12:10 DUR. Bitmese de dur.
12:10 – 12:30 Takılma listesini temize çek → hatalar.md

2.3 Faz 0 çıktıları

- ~/projects/taskr-v0/ — klasör (git yok, commit yok, push yok)
- hatalar.md — takılma kodları + her biri için tek cümle açıklama

taskr-v0 GitHub'a yüklenmeyecek. Bu bir sınav kağıdı, portföy parçası değil. Yerel diskte sakla — Faz 1'i bitirdikten sonra ikisini karşılaştırmak için lazım olacak.

3. Fonksiyonel Şartname

Bu bölüm hem Faz 0 hem Faz 1 için geçerlidir.

3.1 Komutlar

```
taskr add "Alışveriş yap" --priority high --due 2026-08-01
taskr list
taskr list --status pending
taskr list --priority high
taskr done 3
taskr rm 3
taskr stats
```

3.2 Görev nesnesi

Alan	Tip	Kural
id	int	Otomatik artan. Silinen id tekrar kullanılmaz .
title	str	Zorunlu. Boş olamaz.
priority	low / medium / high	Varsayılan: medium
status	pending / done	Varsayılan: pending
created_at	datetime	Otomatik
due_date	date None	Opsiyonel. Format: YYYY-MM-DD
completed_at	datetime None	done komutunda doldurulur

3.3 Kalıcılık

- Konum: ~/.taskr/tasks.json
- Klasör yoksa **otomatik oluşturulacak**
- Dosya yoksa boş liste ile başlanacak (hata verilmeyecek)
- Yazma sırasında mevcut veri kaybolmayacak

3.4 Beklenen çıktı formatları

taskr list

```
ID    PRIORITY STATUS DUE      TITLE
--    -
1     high    pending 2026-08-01 Alışveriş yap
2     medium pending -        Kitap oku
3     low     done    2026-07-20 Faturaları öde
4     high    pending 2026-07-10 ! Rapor teslim et

! = süresi geçmiş
```

taskr stats

```
Total:      4
Completed:  1 (25%)
Pending:    3
Overdue:    1
```

Hata çıktıları — traceback ASLA gösterilmeyecek

```
$ taskr done 99
Error: Task 99 not found.

$ taskr add "Test" --due 01-08-2026
Error: Invalid date format. Expected YYYY-MM-DD.

$ taskr list
Error: Task file is corrupted (~/.taskr/tasks.json).
```

3.5 Zorunlu kenar durumlar

#	Durum	Beklenen davranış
1	Olmayan id ile done / rm	Anlamlı hata mesajı, traceback yok, çıkış kodu 1
2	tasks.json bozuk JSON	Çökme yok, anlamlı hata
3	Geçersiz tarih formatı	Anlamlı hata, beklenen format söylenir
4	Boş başlık ile add	Reddedilir
5	Hiç görev yokken list	Boş tablo değil, "No tasks yet." mesajı
6	Zaten tamamlanmış göreve done	Bilgilendirir, hata vermez

4. Teknik Zorunluluklar

Asıl teşhis burada. Her madde bir körelme alanına karşılık geliyor.

#	Zorunluluk	Ölçtüğü alan
1	<code>argparse</code> ile subcommand yapısı (<code>add_subparsers</code>)	Kod A
2	Görev nesnesi <code>@dataclass</code> , dict değil	Kod B
3	Kendi exception hiyerarşin: <code>TaskrError</code> → <code>TaskNotFoundError</code> , <code>InvalidDateError</code> , <code>CorruptStorageError</code>	Kod C
4	Dosya işlemleri context manager (<code>with</code>) ile	Kod D
5	Yol işlemleri <code>pathlib</code> ile — <code>os.path</code> yok, string birleştirme yok	Kod E
6	Tüm fonksiyon imzalarında type hint	Kod F
7	Çok modüllü paket yapısı (aşağıda)	Kod G
8	<code>datetime</code> / <code>date</code> ayrımı, parse ve karşılaştırma	Kod H
9	<code>dataclass</code> ↔ JSON serileştirme/deserileştirme	Kod I
10	<code>python -m taskr</code> ile çalışması	Kod G

4.1 Zorunlu paket yapısı

```
taskr/
├── taskr/
│   ├── __init__.py
│   ├── __main__.py      # python -m taskr giriş noktası
│   ├── models.py        # Task dataclass, Priority, Status
│   ├── errors.py         # exception hiyerarşisi
│   ├── storage.py        # JSON okuma/yazma, yol yönetimi
│   └── cli.py            # argparse, komut yönlendirme, çıktı formatı
├── tests/                # sadece FAZ 1
│   ├── test_models.py
│   ├── test_storage.py
│   └── test_cli.py
├── pyproject.toml        # sadece FAZ 1
├── README.md             # sadece FAZ 1
├── LICENSE               # sadece FAZ 1 (MIT)
└── SPEC.pdf              # bu belge
```

Katman kuralı: `cli.py` dosya okumaz. `storage.py` ekrana yazmaz. `models.py` ikisini de bilmez. Bu ayırım, sonraki tüm backend işlerinde kullanacağın en temel mimari alışkanlıktır — burada kurulur.

5. Takılma Kodları — Doldurulacak Form

Faz 0 sırasında takıldığın her an ilgili satırı işaretle ve kısa not düş. Bu form, 10 haftalık müfredatının kaynağıdır.

Kod	Alan	"Bunu bilmiyorum" işareti	✓	Notum
A	argparse / subcommand	Subparser kurmayı hatırlamıyorum		
B	dataclass	Nasıl tanımlanır, <code>field(default_factory=...)</code> ne işe yarar		
C	Exception hiyerarşisi	Kendi exception sınıfımı nasıl kurarım, nerede yakalarım		
D	Context manager	<code>with</code> nasıl çalışır, kendim nasıl yazarım		
E	pathlib	<code>os.path</code> refleksine düşüyorsam		
F	Type hints	<code>Optional</code> , <code>list[Task]</code> , <code>Literal</code> yazamıyorsam		
G	Modül / paket	<code>__init__.py</code> , <code>__main__.py</code> , import karışlıyorsa		
H	Tarih / saat	<code>datetime</code> , <code>strptime</code> , <code>date</code> vs <code>datetime</code> bulanıksa		
I	JSON serileştirme	<code>dataclass</code> ↔ JSON dönüşümünde zorlanıyorsam		
J	Veri yapısı tasarımı	"Nereden başlayacağımı bilemedim" dediysem		

5.1 Ek ölçüm

3 saat doldu mu? _____ Kaç komut çalışır durumda? _____ / 6
Kaç kenar durum karşılandı? _____ / 7 Kaç `TODO: bilmiyorum` bıraktım? _____
İşaretlenen kod sayısı: _____ / 10

5.2 Sonuç yorumlama

Durum	Anlamı	Müfredat kararı
3 saatte bitti, 0-2 işaret	Pas ince	10 hafta → 6 haftaya in. Doğrudan SQL ve testle başla.
3 saatte bitti, 3-5 işaret	Beklenen sonuç	Plan aynen uygulanır.
Bitmedi, 6+ işaret	Pas kalın	Hafta 1-2 uzatılır. Panik yok — bunu 2 yıl önce yapabiliyordun, silinmedi.
J işareti var	Tasarım boşluğu	Sorun sözdizimi değil. Müfredat baştan yazılır, mimari ağırlıklı olur.

Hangi sonuç çıkarsa çıksın, bu bir başarısızlık değil. Ölçüm aletinin görevi kötü haber vermek değil, doğru haber vermek. 6+ işaret çıkması, yanlış konuları çalışarak 3 ay kaybetmenden çok daha iyidir.

6. FAZ 1 — Yeniden İnşa

Hafta 1 · Cumartesi (6s) + Pazar (4s) · Yeni klasör: taskr/

taskr-v0'a bakmayacaksın. Aynı şartnameyi sıfırdan uygulayacaksın. Bu sefer Bölüm 4'teki tüm teknik zorunluluklar karşılanacak, testler yazılacak, ve sonuç GitHub'da yayınlanacak.

AI kuralı: Faz 1'de AI'a izin var — ama tek şartla: **açıklayamadığın hiçbir satır commit edilmeyecek.** "Burayı neden böyle yaptın" sorusuna cevabın yoksa, sil ve kendin yaz.

6.1 Cumartesi (6 saat)

09:00 – 09:30 Kağıtta tasarım. Kod yok.
→ Hangi modüller, hangi sorumluluk?
→ Task nesnesinde hangi alanlar?
→ Exception ağacı nasıl?
09:30 – 10:30 Paket iskeleti. `python -m taskr` çalışsın (boş olsa da).
10:30 – 13:00 `models.py` + `errors.py` + `storage.py`
13:00 – 15:00 ARA – planda var, atlanmayacak
15:00 – 17:30 `cli.py`: `add`, `list`, `done`, `rm`, `stats`
17:30 – 18:00 `ruff check` + `mypy --strict` temiz olana kadar. Commit.

6.2 Pazar (4 saat)

07:00 – 07:30 `pytest` kurulumu: `uv add --dev pytest pytest-cov`
07:30 – 09:30 Test yazımı (aşağıdaki liste)
09:30 – 10:00 `pytest --cov` → %70 üstü hedef
10:00 – 11:00 README.md (İngilizce) + LICENSE (MIT)
11:00 – 11:30 GitHub'a public push. Repo adı: taskr

6.3 Zorunlu testler

#	Test	Neyi doğruluyor
1	Görev ekleme artan id üretir	Temel akış
2	Silinen id tekrar kullanılmaz	İş kuralı
3	Olmayan id → <code>TaskNotFoundError</code>	Exception hiyerarşisi
4	Bozuk JSON → <code>CorruptStorageError</code> , çökme yok	Dayanıklılık
5	Geçersiz tarih → <code>InvalidDateError</code>	Girdi doğrulama
6	<code>stats</code> yüzdesi doğru hesaplanıyor	Mantık
7	Geçmiş tarihli görev overdue sayılıyor	Tarih mantığı
8	Boş başlık reddediliyor	Doğrulama
9	Kaydet → oku turu veriyi bozmuyor	Serileştirme
10	Klasör yoksa oluşturuluyor	Kurulum akışı

Test kuralı: `tmp_path` fixture'ını kullan. Testler gerçek `~/taskr/` klasörüne **asla** dokunmayacak. Kendi verisini silen test, kötü testtir.

6.4 README'de bulunması zorunlu bölümler

- Ne yapar** — 2 cümle, İngilizce
 - Kurulum** — kopyalanıp çalıştırılabilir komutlar
 - Kullanım** — her komut için örnek + çıktı
 - Proje yapısı** — modüller ve sorumlulukları
 - Tasarım kararları** — **en önemli bölüm.** Neden dataclass, neden JSON, neden bu exception yapısı, neden bu katman ayrımı
 - Testleri çalıştırma** — komut + coverage bilgisi
5. madde, bu repo'yu diğer on binlerce "task manager" repo'sundan ayıran tek şeydir. Mülakatta okunan bölüm burasıdır.

7. Bitmiş Sayılma Kriterleri

FAZ 0

- [] 3 saat dolduruldu, zamanında durduruldu
- [] Yapay zekâ kullanılmadı
- [] Takılma formu (Bölüm 5) dolduruldu
- [] `hatalar.md` yazıldı
- [] `taskr-v0` GitHub'a **yüklenmedi**

FAZ 1

- [] 6 komutun tamamı çalışıyor
- [] 7 kenar durumun tamamı karşılanıyor
- [] `python -m taskr` çalışıyor
- [] `ruff check` temiz
- [] `mypy --strict` temiz
- [] 10 test geçiyor, coverage %70+
- [] README 6 bölümün tamamını içeriyor
- [] LICENSE (MIT) var
- [] `.gitignore` düzgün — `__pycache__`, `.venv` sızıyor
- [] GitHub'da public, description + topics ekli
- [] Kodun her satırı açıklanabiliyor

8. Bilinçli Olarak Kapsam Dışı

Aşağıdakiler **yapılmayacak**. Kapsam genişletmek bu projenin amacını yok eder ve bitirememe alışkanlığını besler.

- Veritabanı (SQLite dahil) — JSON yeterli, konu o değil
- Web arayüzü, API, GUI
- Etiketler, alt görevler, tekrarlayan görevler, hatırlatıcılar
- Renkli çıktı, TUI, ilerleme çubuğu
- Yapılandırma dosyası, çoklu profil
- Senkronizasyon, bulut, çoklu cihaz

"Şunu da ekleyeyim" dediğin an dur. Fikri `IDEAS.md` dosyasına yaz ve kapat. v1 listesi kapalıdır. Kapsam genişletme, bu plandaki bir numaralı başarısızlık sebebidir.

9. Bittikten Sonra

1. `taskr-v0` ile `taskr`'ı yan yana aç. Farkı gör. Bu, ilerlemenin ilk somut kanıtı.
2. `hatalar.md`'yi 10 haftalık müfredatın olarak kullan — işaretlenen her kod bir haftalık konu başlığı.
3. GitHub profilinde `taskr`'ı sabitle (pinned).
4. Profil README'sindeki `taskr` linki artık gerçek bir repo'ya gidiyor — kırık link kapanır.
5. **İkinci projeye geçmeden önce bu tamamen bitmiş olacak.** Tek aktif proje kuralı geçerli.

Bu proje bittiğinde, 2 yıllık aranın ardından bitirilmiş, test edilmiş ve yayınlanmış ilk işin olacak. Asıl değeri kod değil — bitirme alışkanlığının ilk kanıtı olması.

taskr — Proje Şartnamesi v1.0 · Bu belge `taskr/SPEC.pdf` olarak proje kökünde saklanmalıdır.